

实验一：OpenMP CSR SpMV 并行编程

实验一：OpenMP CSR SpMV 并行编程

问题背景：CSR 稀疏矩阵向量乘

1. 代码结构说明

2. 学习目标

3. 实验要求

3.1 正确性

3.2 OpenMP 并行策略

3.3 `std::thread` 对比

3.4 实验变量与公平性

3.5 Linux 性能工具

3.6 实验指标

3.7 Git 版本管理（可选）

4. 最终提交内容

4.1 代码

4.2 实验报告

4.3 心路历程报告

5. 评分标准

Tips

问题背景：CSR 稀疏矩阵向量乘

稀疏矩阵向量乘（Sparse Matrix-Vector Multiplication, SpMV）计算

$$y = Ax, \quad (1)$$

其中矩阵 A 中绝大多数元素为零。若直接按稠密矩阵存储和计算，不仅浪费内存，也会执行大量无效乘加。CSR（Compressed Sparse Row）只保存非零元素：

- `values[p]`：第 p 个非零值；
- `col_idx[p]`：该非零值所在的列号；
- `row_ptr[i]` 到 `row_ptr[i+1]`：第 i 行的非零元素范围。

第 i 行的计算为：

```
y[i] = sum(values[p] * x[col_idx[p]])  
p = row_ptr[i] ... row_ptr[i+1]-1
```

CSR SpMV 的特点是计算量通常接近 $O(nnz)$ ，但每个非零元素的算术操作很少，性能容易受内存访问、缓存、线程调度和负载均衡影响。不同矩阵的行长度分布也会显著改变并行策略的优劣。

本实验的主要问题是：在给定 CSR 行分布下，如何划分工作、如何选择调度方式、如何用可复现实验验证判断，并解释 OpenMP 与 `std::thread` 各自适合的使用场景。

1. 代码结构说明

本实验基于 `openmp_csr_spmv_lab` starter code。

starter code 当前已经提供：

- CSR 矩阵、输入向量和固定随机种子的生成器；
- 一个串行 CSR SpMV 基线 `spmv_serial`；

- 一个 `std::thread` 连续行分块实现 `spmv_std_thread`;
- 两个 OpenMP 占位函数 `spmv_omp_student_1` 和 `spmv_omp_student_2`;
- 统一的结果正确性检查;
- 基于 `std::chrono::steady_clock` 的重复计时;
- `--rows`、`--cols`、`--normal-nnz`、`--long-rows`、`--long-nnz`、`--threads`、`--repeats` 和 `--seed` 等参数。

代码生成的是一个“多数行很短、少数行特别长”的 skewed CSR 矩阵。

当前主程序只调用两个 OpenMP 函数。若有你觉得特别有趣有意义的方案，可以额外实现第 3--5 个 OpenMP 策略，需要自行增加函数、调用、正确性检查和输出；**函数总数最多为 5 个**，超过 `spmv_omp_student_5` 的部分不计入检查。**额外策略数量不直接加分**。

2. 学习目标

完成实验后，应能够：

1. 正确解释 CSR 数据结构和 SpMV 的工作量；
2. 使用 OpenMP 实现至少两种有明确差异的并行策略；
3. 识别行长度偏斜带来的负载不均衡；
4. 使用 `std::chrono`、加速比、并行效率和 `perf` 做基本性能分析；
5. 解释 OpenMP 和 `std::thread` 在开发成本、控制粒度、线程管理和适用场景上的差异；
6. 使用常用的并行实验指标对结果进行分析；
7. （可选）通过 Git 保留从初始实现到最终优化版本的可追踪过程。

3. 实验要求

3.1 正确性

必须保留串行实现作为参考结果。每个并行策略都必须与参考结果比较，不能只凭“程序运行结束”判断正确。

starter code 的 `check_result` 使用绝对误差和相对误差均为 `1e-10` 的判定。若你修改误差判定、浮点累加顺序或数据类型，必须在报告中说明原因和影响。

禁止通过修改参考结果、跳过检查、只比较部分输出等方式绕过正确性测试。发生数据竞争、未初始化读、越界访问或只在某些线程数下正确的实现，均不视为完成基本要求。

3.2 OpenMP 并行策略

必须实现至少两种不同的 OpenMP 策略。两种策略必须在工作划分、调度方式或并行组织上有实质差异。

最多注册 5 个 `spmv_omp_student_*` 函数。少而精优先：额外实现不会因为数量多而自动加分，重复或没有清晰实验问题的策略可能降低得分。

3.3 `std::thread` 对比

代码已提供 `std::thread` 连续行分块基线。必须至少完成一次 OpenMP 与该基线的定量比较，并讨论：

- 两者是否都适合“按行独立计算”的 SpMV；
- `std::thread` 基线每次调用都会创建并回收线程，这一成本如何影响短任务；
- OpenMP 如何简化并行循环、调度和线程数控制；
- `std::thread` 在需要自定义线程生命周期、任务队列、线程间通信或更细粒度控制的场景中有什么优势；

可以额外实现线程池或改进 `std::thread` 版本，但不是必做项。若修改 baseline，必须保留原始版本或清楚记录修改，并确保比较对象和计时范围公平。

3.4 实验变量与公平性

至少研究以下变量中的三类：

1. 线程数：至少覆盖 `1, 2, 4, ...`，直到机器的物理核心数；
2. 矩阵规模：改变 `rows`、`cols` 或普通行的 `normal-nnz`；
3. 偏斜程度：改变 `long-rows` 和 `long-nnz`；
4. OpenMP 调度：`static`、`dynamic`、`guided` 或 `chunk size`；
5. 随机性：改变随机种子；

至少使用一个消融实验或 **breakdown** 以进行重要结论的支持。请对所有的实验结果进行简单分析，不能只给一张“线程数--时间”图而不解释原因。

计时要求（目前实验代码中已有相应实现）：

- 使用 `std::chrono::steady_clock` 或同等单调时钟；
- 矩阵生成、随机数生成、文件输出和打印不计入 SpMV 核心计时；starter code 已将生成放在 benchmark 外；
- 每个配置重复 10 次实验，建议报告中位数、最小值和最大值，或平均值与离散程度；
- 首次运行可作为 warm-up，不应把明显的首次初始化成本当作稳态性能；
- 记录编译器、编译选项、CPU 型号、核心数、操作系统和 OpenMP 运行时环境。

建议编译命令：

```
g++ -O2 -std=c++17 -fopenmp -pthread csr_spmv_lab.cpp -o csr_spmv_lab
```

若使用 `-O3`、`-march=native` 等选项，必须在报告中写明。不同策略必须使用同一套编译选项。

3.5 Linux 性能工具

至少使用一次 `perf stat` 对串行基线、一个 OpenMP 实现和 `std::thread` 实现进行比较，并对结果进行分析。

由于 starter code 的主程序会在一次运行中顺序测试多个策略，建议增加 `--only` 选项、单独的 benchmark driver，或其他等价方式，使一次 `perf` 运行只测一个目标实现；否则无法把计数器归因到某个策略。示例命令如下：

```
OMP_NUM_THREADS=8 \  
OMP_PROC_BIND=close \  
OMP_PLACES=cores \  
perf stat -r 7 \  
-e task-clock,cycles,instructions,cache-misses,branch-misses,context-switches,cpu-migrations \  
./csr_spmv_one_strategy --strategy omp2 --threads 8 --repeats 10
```

`csr_spmv_one_strategy` 只是示例名称，不是 starter code 自带的可执行文件；你可以通过增加命令行选项、编写小型驱动或复制同一 benchmark 逻辑来实现等价的单策略测量。

建议用 `lscpu`、`uname -a` 和 `perf list` 记录环境。不同 CPU 支持的硬件事件可能不同；事件不可用时，应选择可用替代项并在报告中说明。

可选地使用：

```
perf record -g ./csr_spmv_one_strategy --strategy omp2 --threads 8 --repeats 10  
perf report
```

3.6 实验指标

建议定义：

$$\text{speedup}(p) = \frac{T_1}{T_p}, \quad \text{parallel efficiency}(p) = \frac{\text{speedup}(p)}{p}. \tag{2}$$

作为并行实验指标。

其中 `T_1` 应明确是串行版本时间还是单线程 OpenMP 时间，全文保持一致。

其他可选指标还有latency和throughput。

3.7 Git 版本管理（可选）

推荐使用 Git 管理实验，这样可以保留阶段性提交，便于版本管理和版本回退。报告或心路历程中可以引用 `git log`、`git diff` 或提交哈希。Git 不替代最终提交物，但可作为过程真实性和实验可复现性的证据。

4. 最终提交内容

请提交一个压缩包，包含：

4.1 代码

代码允许使用 AI 辅助，但必须自行检查正确性。AI 生成的代码不构成正确性证明。

4.2 实验报告

允许使用 AI 辅助撰写，但必须由学生核实每条数据和结论。正文 2--4 页，Markdown 或 LaTeX 导出 PDF 均可；超过 4 页的内容放入附录，附录没有义务被阅读，核心结论不能依赖附录。

正文至少包含：

1. 两种或更多 OpenMP 策略的设计说明；
2. 实验环境、参数和公平性控制；
3. 正确性结果（展示实验截图，该截图允许放附录）；
4. 运行时间、加速比和并行效率；
5. OpenMP 策略之间的对比；
6. OpenMP 与 `std::thread` 的简单对比及适用场景；
7. 至少一个消融实验、breakdown 或负优化案例；
8. `perf` 数据及解释；
9. 结论、局限性和可扩展性讨论；
10. AI 使用声明和参考资料链接。

如果有遇到的有趣现象或者bug，也欢迎一并写入报告中。

4.3 心路历程报告

另交 300--1500 字的心路历程报告，格式不限。该部分必须由学生本人完成，不允许 AI 代写或润色；若判定为 AI 辅助撰写，方案决策与探索部分计 0 分。

内容应描述：

- 从最初版本到最终优化方案的思考过程；
- 为什么选择或放弃某些策略；

- 遇到的 bug、数据竞争、错误结果或负优化，以及如何想办法解决；
- 如何设计实验来确认原因并解决问题；
- 哪些想法来自 AI、资料或同学。

如果曾向 AI 询问技术问题，必须如实标明，并附相关对话截图；如果查阅资料，附链接；如果询问同学，注明同学姓名。**资料获取途径本身不作为评分区分，隐瞒来源会影响过程真实性评价。**

5. 评分标准

项目名	分数	说明
基础要求	60	所有提交的策略能通过统一正确性检查，提交的内容齐全
方案决策与探索	30	进行科学实验以辅助方案决策；正确使用性能分析工具；使用合理的实验指标衡量运行结果；实验结果有详细分析
实验结果	10	最终方案相对合理基线具有可信的性能改进，并具有良好的鲁棒性和可扩展性

字数，策略数量本身不作为评分标准，处于要求范围内即可。正文超过 4 页、心路历程超过 1500 字或其他超出规定范围的冗余内容将被扣分；必要的日志、完整表格和补充图放在附录。

Tips

- 评分原则是 **less is more**。策略少但决策科学，分析深入，优于实现很多版本却没有深入解释。
- 过程大于结果，多和其他同学交流探讨。